TECHNISCHE HOCHSCHULE MITTELHESSEN

CAMPUS GIESSEN

MNI
Mathematik, Naturwissenschaften und Informatik

Datenbanken und Informationssysteme
Spaltenorientierte Datenbanken

Ein Vorlesungsskript von

Pia Georgiew

Dozent: Markus Siepermann

Inhaltsverzeichnis

1 Einführung 1

2 Kontext 3
- 2.1 OLTP und OLAP . . . 3
- 2.2 NoSQL-Datenbanken . . . 3
- 2.3 CAP-Theorem . . . 4
- 2.4 Wide Column Stores . . . 5

3 Eigenheiten der spaltenorientierten Datenbanken 7
- 3.1 Speicher- und Einlesestrukturen . . . 7
- 3.2 Parallelisierung . . . 8
  - 3.2.1 Distribution- und Sortkeys . . . 8
  - 3.2.2 Map-Reduce . . . 9
- 3.3 Vor- und Nachteile . . . 9
- 3.4 Performancetest . . . 10

4 Vergleich verschiedener Datenbanken 13

5 Zusammenfassung 15

Literaturverzeichnis 17

Abbildungsverzeichnis 19

1 Einführung

Die International Data Corporation hat berechnet, dass bis 2025 rund 175 Zettabyte an Daten gespeichert werden müssen. [Rei] Viele dieser Daten sollen ausgewertet werden. Eine Speicherart, die sich besonders gut für das Anwendungsgebiet der Datenanalyse eignet, ist die spaltenorientierte Datenbank (DB). In diesen werden Einträge spaltenweise gespeichert.

ID	Last Name	First Name	Point
5	Miller	Ginny	600
23	Parker	Peter	87
6	Stacy	Gwen	250

Diese Tabelle Beispielsweise, würde in einer spaltenorientierten DB folgendermaßen abgespeichert werden:

5, 23, 6; Miller, Parker, Stacy; Ginny, Peter, Gwen; 600, 87, 250

Durch die spaltenorientierte Speicherung können Operationen wie Summierungen oder Rankings mit wenigen Lesezugriffen berechnet werden. Auch die Anzahl der unnötig eingelesenen Daten kann, im Vergleich zu traditionellen zeilenorientierten DB, verringert werden.

Im folgenden Kapitel wird zuerst eine Kontexteinordnung stattfinden. Danach wird auf die Eigenheiten von spaltenorientierten DB eingegangen. In Kapitel 4 werden danach noch verschiedene Datenbanken verglichen.

2 Kontext

2.1 OLTP und OLAP

Datenbankanwendungen können häufig in zwei Bereiche unterteilt werden. Es gibt den Bereich des „Online Transactional Processing“, kurz OLTP und den Bereich des OLAP („Online Analytical Processing“). In OLTP Anwendungen werden Echtzeit-Transaktionen von vielen verschiedenen Nutzern ausgeführt. Häufig werden nur wenige Einträge aus einer Tabelle mit vielen Attributen abgefragt oder geschrieben. OLTP Systeme verwenden häufig relationale Datenbanken, also einen zeilenorientierten Ansatz. Im Gegensatz dazu steht der OLAP Ansatz für analytische Anwendungszwecke. Dabei soll es dem Anwender ermöglicht werden, Theorien zu seiner Datendomäne anhand der vorliegenden Daten zu überprüfen. Die Analysen können dann zur Entscheidungsunterstützung dienen. Um dies umzusetzen, kann als Datenstruktur ein OLAP-Würfel verwendet werden. Dieser folgt einer mutidimensionalen, datenpunktorientierten Logik. [OLA] Aber auch spaltenorientierte DB eignen sich für OLAP. So ist zum Beispiel der Data-Warehouse-Service Amazon Redshift, eine spaltenorientierte, massiv parallele Datenbank. [Spa]

2.2 NoSQL-Datenbanken

Die spaltenorientierten DB werden den Nicht-Relationalen DB oder NoSQL-DB zugeordnet. NoSQL steht in diesem Kontext für Not Only SQL. Zu den NoSQL-DB gehören des Weiteren auch Formate wie dokumentenorientierte Datenbanken oder Graphdatenbanken.

Die Sprache SQL wurde zum Arbeiten mit relationalen DB entwickelt und eignet sich für viele Formen der Nicht-Relationalen DB nicht. Dies ist unter anderem darin begründet, dass in relationalen DB für jede Tabelle ein festes Schema definiert werden muss, welches die Organisation der Daten beschreibt. NoSQL-DB haben häufig kein festes Schema, was die Nutzung flexibler macht. Spaltenorientierte DB bilden hier eine Ausnahme, da es einige DB gibt die ein festes Schema voraussetzen. Dies führt dazu, dass in diesen Datenbanken dann auch SQL oder an SQL angelehnte Abfragesprachen genutzt werden

können. Beispiele für solche Datenbanken sind Amazon Redshift und Apache Cassandra. [Kha] [Was]

2.3 CAP-Theorem

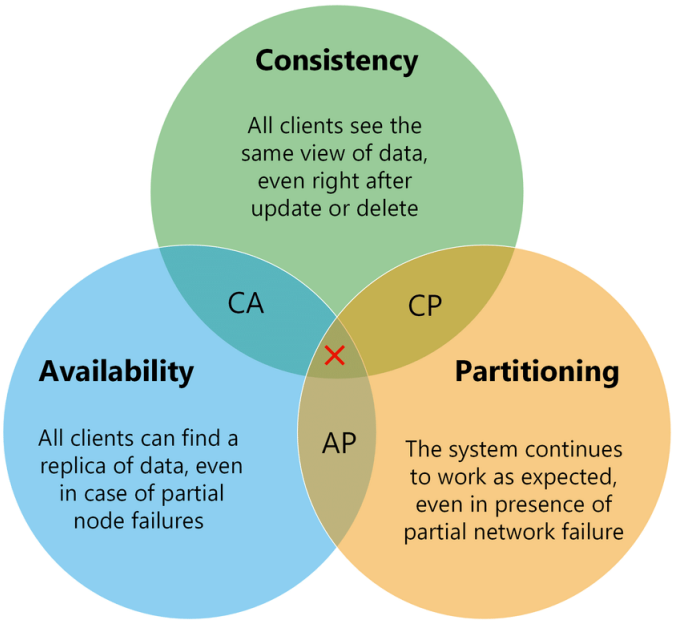

Abbildung 2.1: Erklärung des CAP-Theorems [Gho]

Das CAP-Theorem besagt, dass es in einem verteilten System, also auch in spaltenorientierten DB, unmöglich ist alle drei Eigenschaften Konsistenz, Verfügbarkeit und Ausfalltoleranz zu gewährleisten. Es muss sich für zwei der drei Eigenschaften entschieden werden.
Da es sich um verteilte Systeme handelt, ist eine Auswahl ohne Ausfalltoleranz nicht Teil der Betrachtung (das wird vom CAP-Theorem so vorgegeben). Die Entscheidung liegt also darin, dass im Falle einer Netzwerkpartionierung oder einem Fehlerfall zwischen der Konsistenz der Daten und der Verfügbarkeit entschieden werden muss.
Da Konsistenz für viele Nutzer besonders wichtig ist, wird sich oft für die Kombination aus Konsistenz und Ausfalltoleranz entschieden. [DLE] Das bedeutet, dass es im schlimmsten Fall dazu kommen kann, dass das System nicht mehr antwortet, wenn die Kommunikation zwischen den Nodes nicht stattfinden kann.
Bei der Kombination von Verfügbarkeit und Ausfalltoleranz antwortet das System immer. Es kann aber in Ausnahmefällen vorkommen, dass die Compute Nodes nicht die aktuellen Daten haben und es so zu abweichenden Antworten kommt. [MM]

2.4 Wide Column Stores

Obwohl die Wide Column Stores auch zu den spaltenorientierten DB gehören, unterscheiden sie sich stark von den spaltenorientierten DB. In Wide Column Stores werden Zeilen durch einen Schlüssel identifiziert. Eine Zeile kann eine variable Anzahl an Spalten enthalten. Eine Menge an Zeilen wird zu einer Column Family zusammengefasst und diese werden dann in einem Keyspace organisiert. Eine Menge an Keyspaces inklusive dem System-Keyspace bilden ein Datenbanksystem.
Mit dieser Struktur ähneln die Wide Column Stores eher den Key-Value Datenbanken als den Spaltenorientierten Datenbanken. Trotzdem gehören sie organisatorisch zu den spaltenorientierten DB. [Sem] [Kha]

3 Eigenheiten der spaltenorientierten Datenbanken

3.1 Speicher- und Einlesestrukturen

In OLAP Systemen müssen meist wenige Attribute von vielen Datentupeln (auch Einträge genannt) gelesen werden. Die spaltenorientierten DB sind durch ihre Speicherstruktur darauf ausgelegt, solche Anfragen mit möglichst wenigen Input/Output-Operationen zu beantworten. Denn es liegen alle Einträge eines Attributs in einer Spalte zusammen. Die Spalten werden in Datenblöcken abgespeichert. Übliche Datenblockgrößen liegen zwischen 2 KB bis 32 KB. Amazon Redshift nutzt sogar eine Blockgröße von 1 MB. [Spa]
Alle Attribute eines Datentupels liegen am selben Indize und können so zugeordnet werden. Da das Aktualisieren vieler Attribute eines Tupels relativ viel Aufwand mit sich bringt, kann nach dem „Append only style“ gearbeitet werden. Statt Zeilen zu aktualisieren, wird eine neue Zeile mit einer aktualisierten Gültigkeit an die Tabelle angefügt. Dadurch wird ein Reordering oder Encoding vermieden. [Far] Da, anders als beim zeilenorientierten Ansatz, alle Einträge denselben Datentyp haben, können die Daten weiter komprimiert werden. Eine Kompression kann dann zum Beispiel so aussehen, dass Folgen von identischen Werten lauflängencodiert abgelegt werden. Das bedeutet, dass statt für jede Zeile der Wert abgespeichert wird, nur die Anzahl an aufeinanderfolgenden gleichen Werten und die Ausprägung am Spaltenanfang gespeichert wird.
Als Beispiel aus 0000 0111 wird dann 5 3.
Auch können mehrfach vorkommende Werte durch Kürzel ersetzt werden und mit einem Wörterbuch bei Bedarf wieder in die ursprünglichen Werte übersetzt werden. Es gibt noch viele weitere komplizierte Möglichkeiten den benötigten Speicherplatz zu reduzieren. [fG]
Speicherplatz soll aber nicht nur auf der Festplatte, sondern auch im Arbeitsspeicher so gut wie möglich reduziert werden. Daher werden nur benötigte Blöcke mit benötigten Spalten abgerufen. Bei Anfragen, bei denen nur wenige Attribute vieler Tupel gebraucht werden, müssten in relationalen Datenbanken alle Tupel vollständig eingelesen werden, was eine deutliche größere Anzahl an Input/Output-Operationen zur Folge hat. Bei einer Tabelle mit 100 Spalten und einer Abfrage die nur fünf Spalten benötigt, müssen nur etwa fünf Prozent der in der Tabelle enthaltenen Daten gelesen werden. In zeilenori-

entierten DB müssten alle Spalten eingelesen werden. Diese Effizienzgewinne werden bei einer sehr hohen Anzahl an Spalten und Zeilen proportional größer. [Spa]

3.2 Parallelisierung

Zur schnellen Verarbeitung von großen Datenmengen wird eine Möglichkeit der parallelen Datenverarbeitung benötigt. Dafür wird eine Tabelle in mehrere Segmente geteilt und diese dann je auf einem Knoten ausgeführt. Jeder Knoten bearbeitet dieselbe Query. Nach Bearbeitung werden die Ergebnisse aggregiert und zurückgegeben.
Es gibt viele unterschiedliche Konzepte, diese Verteilung umzusetzen. Amazon Redshift verwendet Distribution- und Sortkeys, während Apache Cassandra und HBase auf Map-Reduce setzen.

3.2.1 Distribution- und Sortkeys

Um eine optimale Verteilung der Daten zu erreichen, kann für jede Tabelle ein distribution Style gewählt werden. Konkret wird eine Spalte ausgewählt, nach der die Daten verteilt werden. Das Ziel dabei ist es, die benötigten Daten nur einmal auf die Knoten zu verteilen und damit eine Query vollständig bearbeiten zu können. Das Nachladen von Daten soll vermieden werden. Außerdem soll die Rechenlast auf allen Knoten ungefähr gleichverteilt werden. Wenn zwei Tabellen also zum Beispiel häufig zusammen gejoined werden, macht es Sinn, die beiden Join-Schlüssel, als DISTKEY zu setzen. Dadurch können die Daten so verteilt werden, dass jeweils die Teile, die vereint werden müssen, auf einem Knoten liegen. [Choa]
Eine weitere Möglichkeit, Informationen über die Nutzung einer Tabelle zur schnelleren Verarbeitung anzugeben, ist der SORTKEY. Dieser legt fest, nach welcher Systematik eine Tabelle sortiert werden sollte. Wenn die neuesten Daten am häufigsten benutzt werden, macht es Sinn, eine Spalte mit Zeitstempel als Sortkey festzulegen. Bei häufigem Filtern bietet es sich an die Spalte, auf der am meisten gefiltert wird, als Sortkey festzulegen. Wenn stattdessen häufig Join-Operationen verwendet werden, macht es Sinn die Join-Spalte zusätzlich auch als Sortkey zu setzen. Ein gesetzter Sortkey kann dabei helfen, bestimmte Blöcke von Daten bei der Verarbeitung überspringen zu können. Denn zu jedem Block wird der minimale und maximale Wert gespeichert. Wird zum Beispiel als Filter gesetzt, dass die Werte größer als 12 sein müssen, könnten Blöcke, die einen maximalen Wert unter 12 haben, direkt übersprungen werden. [Chob]

3.2.2 Map-Reduce

Eine weitere Methode zur Parallelisierung ist das Konzept des Map-Reduce. Im Schritt des Mappings wird zu einem Inputwert ein Outputwert berechnet. Im zweiten Schritt, dem Reduce, werden die Outputs des Mappings zu einem Wert zusammengefasst. Dieser Ablauf ist in Abbildung 3.1 dargestellt. Durch dieses Vorgehen können Anfragen auf verschiedene Recheninstanzen verteilt werden. [Lah]

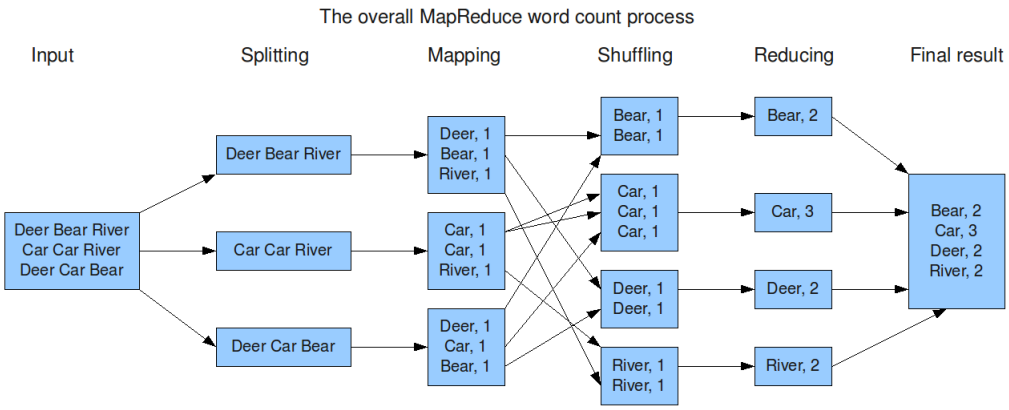

Abbildung 3.1: Konzept des Map-Reduce [Map]

3.3 Vor- und Nachteile

Vorteile:

- Performant bei OLAP Anwendungen (Data Analytics, Entscheidungsunterstützung)
  - Leistungstark im Umgang mit großen Datenmengen
  - Parallelisierung (horizontale Partionierung, Map-Reduce, verteilte Datenbankinstanzen)
  - Fensterfunktionen (Min, Max, Average, Rankings)
- nur tatsächlich benötigte Spalten werden eingelesen
- Effiziente Datenkompression
- Schnelles Lesen (z.B. Abfrage von mehreren Einträgen in einer Spalte in einer Output Operation (Disk IO))

Nachteile:

- Langsam beim Einfügen und Bearbeiten von Einträgen
- Langsam bei OLTP Anwendungen (Kundenorientiert, viele Transaktionen)
- Langsam bei häufigen JOIN-Operationen

[Sor] [Far]

3.4 Performancetest

Eine wichtige Eigenschaft von Datenbanken besteht aus der Performance. Diese ist von Faktoren wie den verfügbaren Rechenressourcen und dem Arbeitsspeicher abhängig. Unterschiedliche Datenbanken sind außerdem auf verschiedene Abfragearten optimiert. In dem Paper „Performance of columnar databases“ [MM] wurden verschiedene DB verglichen. Es wurden drei verschiedene Abfragen genutzt, um die Unterschiede deutlich zu machen.
Query 1:

```
SELECT id, item_nbr, store_nbr, date
FROM train
WHERE id = 500023352;
```

Query 2:

```
-- Returns 30% of the total data
SELECT id, item_nbr, store_nbr, date
FROM train
WHERE date >= '2015-12-31';
```

Query 3:

```
SELECT SUM (unit_sales)
FROM train
WHERE store_nbr = '12'
```

In der Abbildung 3.2 ist zu erkennen, dass bei der Abfrage eines Wertes aus einer Tabelle mit einer Million bis 50 Millionen Einträgen MySQL, eine relationale Datenbank, mit ungefähr 2 Millisekunden Antwortzeit am schnellsten war. Die im Vergleich betrachteten

spaltenorientierten DB MonetDB und Cassandra konnten ebenfalls eine schnelle Antwort geben. Spannend ist auch zu beobachten, dass die Antwortdauer sich bei mehr Einträgen in allen Datenbanken kaum bis gar nicht erhöht.

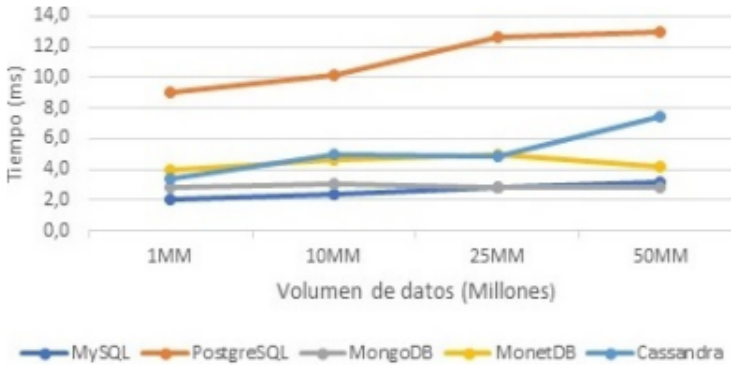

Figura 7. Results first query

Abbildung 3.2: Performancevergleich bei Abfrage eines Eintrags [MM]

In der zweiten Query wurden 30 Prozent der gesamten Einträge ausgegeben. In Abbildung 3.3 ist direkt zu erkennen, dass, bei den relationalen Datenbanken, die Abfragedauer mit der Anzahl der Datensätze wächst. Die dokumentenorientierte Datenbank MongoDB und die spaltenorientierten DB antworten sehr schnell und die Abfragedauer ist gleichbleibend.

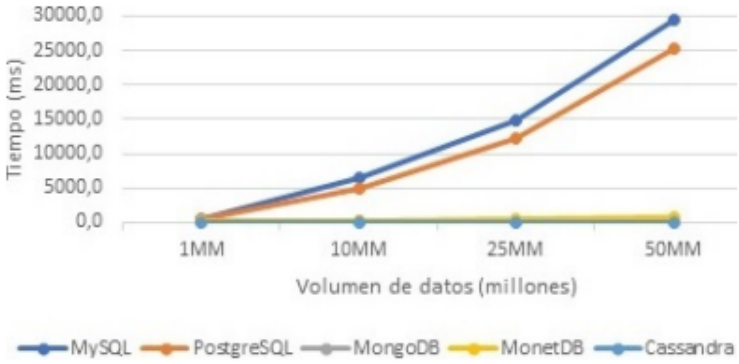

Figura 8. Results second query

Abbildung 3.3: Performancevergleich bei Abfrage von 30 Prozent der Gesamtdaten [MM]

Die Ergebnisse der dritten Abfrage (3.4) zeigen, dass bei einer Summierung eine Erhöhung der Datenanzahl zu längeren Antwortzeiten führt. Bei spaltenorientierten DB ist dieser Anstieg deutlich geringer als bei den relationalen DB. Bei 50 Millionen Einträgen brauchen die spaltenorientierten DB nur etwa ein Viertel der Antwortzeit.

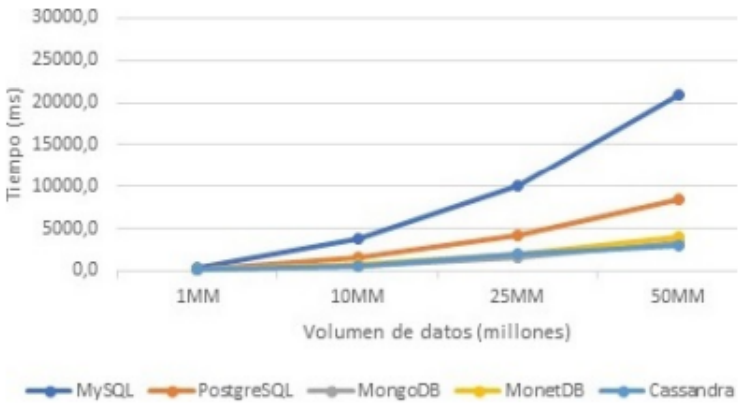

Figura 9. Results third query

Abbildung 3.4: Performancevergleich bei einer Summierung [MM]

4 Vergleich verschiedener Datenbanken

TABLE 2 COLUMN ORIENTED TYPES AND CHARACTERISTICS [11][14]

	PERSISTENCE	REPLICATION	SHARDING	CONSISTENCY	API	QUERY METHOD	IMPLEMENTATION LANGUAGE	CAP
HBASE	built on top of HDFS	Handled on HDFS	By key value, it allows auto sharding	High consistency	Java REST-ful Thrift Avro	MapReduce	Java	CP
HYPERTABLE	HDFS	HDFS	Key-value Allows chunking	High consistency	C++ API Java REST-ful Thrift Avro	HQL (Hypertable query language), MapReduce	C++	CP
CASSANDRA	Proprietary format	Auto partition with zero loss	By row hashing	Eventually consistent	CQL Thrift	MapReduce	Java	AP
ACCUMULO	HDFS	Multi master replication	By row	Eventually consistent	Java REST-ful Thrift	MapReduce	Java	CP

Abbildung 4.1: Vergleich von spaltenorientierten Datenbanken [Kha]

Je nach Anwendungsfall sollte die passendste Datenbank gewählt werden. In dem Paper von Tariq N. Khasawneh, Mahmoud H. AL-Sahlee und Ali A. Safia „SQL, NewSQL, and NOSQL Databases: A Comparative Survey“ wurden unter anderem verschiedene Open-Source verfügbare spaltenorientierte DB verglichen (Abbildung 4.1).
Die meiner Meinung nach wichtigsten betrachteten Kategorien sind Sharding, Consistency, Query Method und CAP. In der Kategorie Sharding wird gezeigt, nach welchem Verfahren das Verteilen der Daten auf die Compute-Knoten abläuft. Die Consistency beschreibt die Datenkonsistenz auf den Knoten. Bei „High consistency“ werden Daten immer als erstes auf alle Shards verteilt, bevor Anfragen ausgeführt werden. Bei „Eventually consistency“ findet diese Verteilung asynchron statt. Es kann zu geringen Verzögerungen kommen. Die Query Method beschreibt die Benutzerschnittstelle und CAP das bereits behandelte CAP-Theorem.
Die erste DB ist HBase, welche auf HDFS (Hadoop Distributed File System) aufbaut. HDFS ist ein System zur Speicherung und Verwaltung von Dateien, die über mehrere Knoten hinweg in einem Netzwerk verteilt liegen. HBase ist vor allem für seine Echtzeitverarbeitung und den Umgang mit großen Tabellen bekannt. [Apab]
Die nächste vorgestellte Datenbank ist Hypertable, welche ebenfalls auf HDFS basiert und Google Big Table nachempfunden ist. Hypertable ist besonders stark im Bereich

der Skalierbarkeit und der Performance. Tabellen werden mittels Key-Value Paaren eingeteilt und partitioniert.
Cassandra ist eine NoSQL-Datenbank, die für verteilte und hoch skalierbare Anwendungen entwickelt wurde. Cassandra bietet eine schnelle und effiziente Datenzugriffsleistung. Dies wird durch die Verteilung der Daten über ein Cluster und die Verwendung eines verteilten Hashing-Algorithmus erreicht.
Die letzte im Paper vorgestellte DB ist Accumulo. Auch Accumulo basiert auf HDFS und zeichnet sich durch ein besonders gutes und feingranulares Rechtemanagement aus. [Apaa] Alle vier DB zählen zu den Wide Column DB.
Zwei weitere bekannte spaltenorientierte DB sind SAP Hana und Redshift. Die verglichenen Eigenschaften können der folgenden Tabelle entnommen werden.[SAP] [Cre] [AWS]

	Sharding	Consistency	Query Method	CAP
SAP Hana	Range Partitioning, Hashing	High Consistency	SQL, Calculation-/ Analytic-Views	CP
Redshift	Distribution Keys	Eventually Consistency	SQL	AP

5 Zusammenfassung

Bei spaltenorientierten DB werden Einträge einer Spalte aufeinanderfolgend in Datenblöcken abgespeichert. Diese können komprimiert werden, um Speicherplatz zu sparen. Beispiele für gängige Komprimierungsverfahren sind die Lauflängencodierung oder der Einsatz von Kürzeln und Wörterbüchern. Im Großen und Ganzen besteht das Ziel immer darin, bei der Verarbeitung einer Anfrage möglichst wenige Input/Output-Operationen zu benötigen.
Spaltenorientierte DB sind hervorragend im OLAP-Kontext einsetzbar. Sie sind unter anderem auf die Ausführung von Fensterfunktionen wie Summierungen oder Rankings ausgelegt. Auch große Datenmengen können schnell verarbeitet werden, indem vor allem die Parallelisierung durch den Einsatz verschiedenster Verteilmechanismen wie Map-Reduce oder das setzen von Distributionkeys, optimiert wurde. Neben der Parallelisierung können verteilte Datenbanken zwei der drei Eigenschaften des CAP-Theorems (Konsistenz, Verfügbarkeit und Partitionierung) umsetzen.
Beispiele für spaltenorientierte DB sind SAP Hana, Redshift, MariaDB ColumnStore und die zur Familie der Wide Column Stores gehörenden DB HBase, Cassandra und Accumulo. Jede dieser DB setzt unterschiedliche Schwerpunkte und muss Anwendungsbezogen ausgewählt werden.
Spaltenorientierte DB, welche zu den NoSQL DB gehören, bilden einen Teil der DB-Familien, die bis 2025 rund 175 Zettabyte an Daten speichern werden. [Rei] Es bleibt also abzuwarten, was für spannende Neuerungen uns in Zukunft noch erwarten.

Literaturverzeichnis

[Apaa] Apache Accumulo, URL https://accumulo.apache.org/

[Apab] Apache HBase – Apache HBase™ Home, URL https://hbase.apache.org/

[AWS] AWS | Amazon Data Warehouse Redshift – Datenverwaltung, URL https://aws.amazon.com/de/redshift/

[Choa] Choose the Best Distribution Style - Amazon Redshift, URL https://docs.aws.amazon.com/redshift/latest/dg/c_best-practices-best-dist-key.html

[Chob] Choose the Best Sort Key - Amazon Redshift, URL https://docs.aws.amazon.com/redshift/latest/dg/c_best-practices-sort-key.html

[Cre] Create a Calculation View, URL https://developers.sap.com/tutorials/hana-cloud-mission-trial-8.html

[DLE] D. L. Engle, Ryan; T. Langhals, Brent; R. Grimaila, Michael und D. Hodson, Douglas: Evaluation Criteria for Selecting NoSQL Databases in a Single Box Environment. Bd. 10(4): S. 01–12, URL http://aircconline.com/ijdms/V10N4/10418ijdms01.pdf

[Far] Farber, Franz; May, Norman; Lehner, Wolfgang; Grosse, Philipp; Rauhe, Hannes; Dees, Jonathan und Muller, Ingo: The SAP HANA Database – An Architecture Overview

[fG] family=Informatik (GI), prefix=für useprefix=false, given=Gesellschaft: Spaltenorientierte Datenbank, URL https://gi.de/informatiklexikon/spaltenorientierte-datenbank

[Gho] Ghosh, Abhishek: CAP Theorem Explained, URL https://thecustomizewindows.com/2021/01/cap-theorem-explained/

[Kha] Khasawneh, Tariq N.; AL-Sahlee, Mahmoud H. und Safia, Ali A.: SQL, NewSQL, and NOSQL Databases: A Comparative Survey, in: *2020 11th International Conference on Information and Communication Systems (ICICS)*, S. 013–021

[Lah] Lahteenmaki, Peitsa: MapReduce With Columnar Storage

[Map] MapReduce [Wirtschaftsinformatik Wiki - Kewee], URL https://wi-wiki.de/doku.php?id=bigdata:mapreduce

[MM] Morales-Morales, Mario Raul; family=Fenster), prefix=in neuem useprefix=false, given=Link zu externer Website (Anzeige; Durán-Cazar, Jhonatan W.; family=Fenster), prefix=in neuem useprefix=false, given=Link zu externer Website (Anzeige; Tandazo-Gaona, Eduardo J.; family=Fenster), prefix=in neuem useprefix=false, given=Link zu externer Website (Anzeige; Cardoso, Santiago Morales und family=Fenster), prefix=in neuem useprefix=false, given=Link zu externer Website (Anzeige: Performance of Columnar Database. (22): S. 47–58, URL https://www.proquest.com/docview/2262078495/abstract/ABABD5DCFA7E4181PQ/1

[OLA] OLAP vs. OLTP: What's the Difference? | IBM, URL https://www.ibm.com/cloud/blog/olap-vs-oltp

[Rei] Reinsel, David; Gantz, John und Rydning, John: The Digitization of the World from Edge to Core

[SAP] SAP Developer Center | Official Resources for SAP Developers, URL https://developers.sap.com/

[Sem] Seminar Cloud-Computing (SS2012), URL https://www.christianbaun.de/SEM12/index.html

[Sor] Sorjonen, Sami: OLAP Query Performance in Column-Oriented Databases (December 2012)

[Spa] Spaltenweise Speicherung - Amazon Redshift, URL https://docs.aws.amazon.com/de_de/redshift/latest/dg/c_columnar_storage_disk_mem_mgmnt.html

[Was] Was ist NoSQL? – Amazon Web Services (AWS), URL https://aws.amazon.com/de/nosql/

Abbildungsverzeichnis

2.1 Erklärung des CAP-Theorems [Gho] 4

3.1 Konzept des Map-Reduce [Map] 9
3.2 Performancevergleich bei Abfrage eines Eintrags [MM] 11
3.3 Performancevergleich bei Abfrage von 30 Prozent der Gesamtdaten [MM] 11
3.4 Performancevergleich bei einer Summierung [MM] 12

4.1 Vergleich von spaltenorientierten Datenbanken [Kha] 13